Part 2: communicating with data

Lino Galiana
Version of November 2, 2026

Data scientists need to be able to synthesize the information contained in a dataset through graphical representation, because the human brain understands information better through figures than through tables. Data visualization is important both as part of an exploratory approach to understanding the structure of the phenomena under study, but also as part of a phase of communicating results to audiences who don't necessarily have access to raw data and need to make do with summaries. This part of the course is an introduction to this vast subject through the practical construction of descriptive graphs and maps.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **2**
1.1 Data visualization, an essential part of communication work 2
1.2 The role of visualization in the data value creation process 2
2 Communicating, an opening to *data storytelling* **3**
3 Communicating, an opening to app development **5**
4 The `Python` ecosystem **5**
4.1 Data visualization packages 5
4.2 Visualization applications 6
4.3 Summary of this section 7
4.4 Useful references 7
Informations additionnelles 8
Bibliography **9**

Skills you will acquire in this chapter

- Recognize the value of data visualization for both exploring data and communicating insights
- Understand how visualization helps transform raw data into clear, compelling messages
- Be aware of the cognitive principles behind effective visuals—making complex data easier to grasp and more engaging
- Get familiar with the main categories of Python visualization libraries:
  - static plotting libraries (e.g. Matplotlib, Seaborn, Plotnine)
  - interactive libraries (e.g. Plotly, Folium)
- Practice recreating real-world charts (e.g. from Paris open data) to better understand design decisions in context
- Learn the fundamentals of visual communication: choose the right chart type (distribution, correlation, time series, map, etc.) and focus on the message you want to convey

1 Introduction

An essential part of the work of a *data scientist* is to synthesize the information contained in their datasets in order to distinguish what constitutes the signal, which they can focus on, and what constitutes the noise inherent in any dataset. In the work of a *data scientist*, during an exploratory phase, there is a constant back-and-forth between synthesized information and disaggregated datasets. It is therefore essential to know how to synthesize the information in a dataset before grasping its structure, which can then guide further analyses, whether for a modeling phase or data correction (anomaly detection or bad data retrieval).

We have already explored a key part of this work, namely the construction of relevant and reliable descriptive statistics. However, if we were content to present information using raw outputs from the `groupby` and `agg` combo on a `Pandas` *DataFrame*, our understanding of the data would be quite limited. The implementation of stylized tables using `Great Tables` was already a step forward in this process but, in truth, our brain processes information much more intuitively through simple graphical visualizations than through a table.

1.1 Data visualization, an essential part of communication work

As humans, our cognitive capacities are limited, and we can only grasp a limited amount of information, whereas computers are capable of processing large volumes of information. For a *data scientist*, this means that using our computational and statistical skills to obtain synthetic representations of our many datasets is essential to meet operational or scientific needs. The range of methods and tools that make up the toolbox of *data scientists* aims to simplify the understanding and subsequent exploitation of datasets whose volume exceeds our cognitive capacities.

This brings us to the question of data visualization, a set of tools and principles for representing stylized facts or contextualizing individual data in a synthetic manner. Data visualization is the art and science of **visually representing complex and abstract information through visual elements**. Its primary goal is to synthesize the information contained in a dataset to facilitate the understanding of its key issues for further analysis. Data visualization allows, among other things, to highlight trends, correlations, or anomalies that might be difficult or even impossible to grasp just by looking at raw data, which requires some context to make sense of it.

Data visualization plays a crucial role in the data analysis process by providing visual means to explore, interpret, and communicate information. It facilitates communication between data experts, decision-makers, and the general public, enabling the latter to benefit from the rigorous work of the former to make sense of the data without the need for deep conceptual knowledge that underpins the synthesized information.

1.2 The role of visualization in the data value creation process

Data visualization is not limited to the final phase of a project, which is the communication of results to an audience that does not have access to the data or the means to make use of it. Visualization plays a role at every stage of the data value creation process. It is, in fact, an essential part of the process of transitioning from a record, a snapshot of a phenomenon, to data— a record that has value because it carries information on its own or when combined with other records.

The daily work of a *data scientist* involves examining a dataset from every angle to identify key value extraction opportunities. Quickly knowing what statistics to represent, and how, is crucial for saving time during this exploratory phase. This is primarily a form of self-communication that can

afford to be rough around the edges, as the goal is to sketch the work before refining certain aspects. The challenge at this stage of the process is not to overlook any dimension that could potentially bring value.

The truly time-consuming communication work comes when presenting to an audience with limited data access, unfamiliar with sources, with a limited attention span, or without quantitative skills. These audiences cannot be satisfied with raw outputs like a *DataFrame* in a *notebook* or a graph created in seconds with the `plot` method from `Pandas`. It is important to adapt to their evolving expectations, and the tools they are familiar with, which explains the growing importance of websites dedicated to *data visualizations*.

2 Communicating, an opening to *data storytelling*

Data visualization thus holds a special place among the various techniques of *data science*. It is involved at all stages of the data production process, from upstream (exploratory analysis) to downstream (presenting results to various audiences), and when well-constructed, it allows us to intuitively grasp the structure of the data or the key issues of its analysis.

As an art of synthesis, data visualization is also the art of storytelling, and when done well, it can even reach the level of artistic production. *Data visualization* is a profession in its own right, with more and more practitioners found in media outlets or specialized companies (`Datawrapper`, for example).

Without aiming to create visualizations as sophisticated as those produced by specialists, every *data scientist* should be able to quickly generate visualizations that synthesize the information in the datasets at hand. A clear and readable visualization, while remaining simple, can be more effective than a speech in conveying a message.

Just like a speech, a visualization is a form of communication in which a speaker—the person constructing the visualization— seeks to convey information to a recipient—potentially the same person as the speaker since a visualization can be created for oneself during exploratory analysis. It is no surprise that during the period when semiology played a significant role in intellectual debates, especially around the figure of Roland Barthes, the concept of graphic semiology emerged, centered around Jacques Bertin [1], [2]. This approach allows reflection on the relevance of the techniques used to convey a graphic message, and many visualizations, if they followed some of these rules, could be improved at little cost.

Eric Mauvière, a French statistician and a successor to Bertin's school of graphic semiology, offers excellent content on the subject. Some of his presentations, notably the one for `SSPHub`, presented in the Note 1, should be viewed in all *data science* training programs as they highlight the numerous pitfalls encountered by *data scientists*.

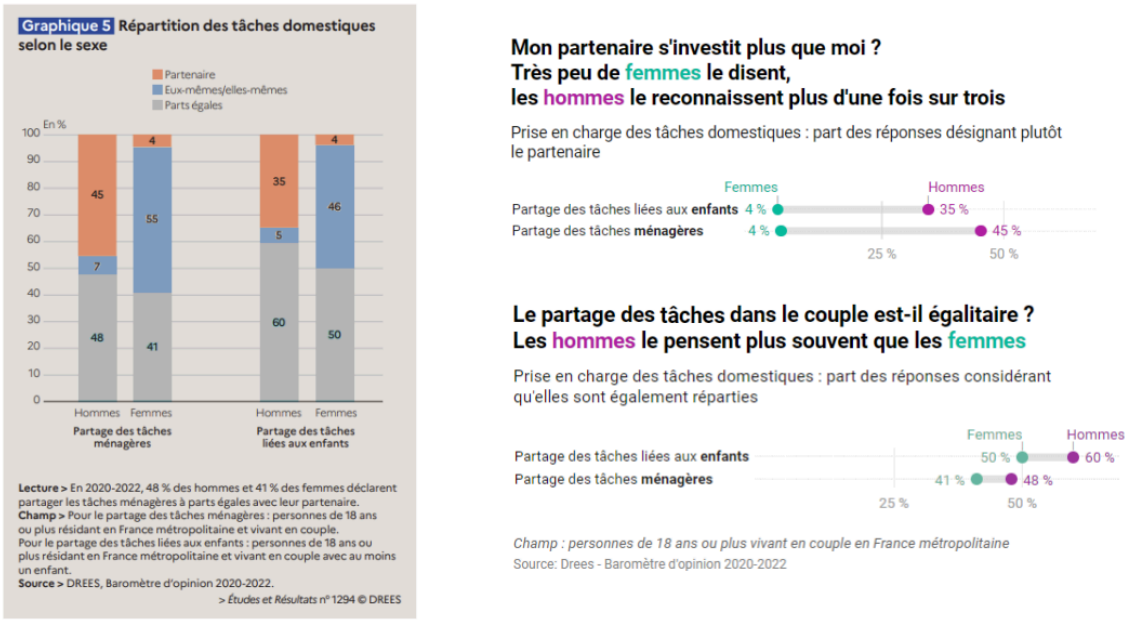


Figure 1: An example of two visualizations made from the same dataset by Eric Mauvière, see **?@nte-mauviere**

ℹ Note 1: A conference by Eric Mauvière on the subject

```
//| echo: false
html`${slides_button}`
```

https://minio.lab.sspcloud.fr/lgaliana/ssphub/replay/20240229-dataviz-mauviere/video1991622347.mp4

```
//| echo: false
slides = "https://minio.lab.sspcloud.fr/lgaliana/ssphub/replay/20240229-dataviz-mauviere/conf_ssphub_icem7.pdf"
```

```
//| echo: false
slides_button = html`<p class="text-center">
  <a class="btn btn-primary btn-lg cv-download" href="${slides}" target="_blank">
    <i class="fa-solid fa-file-arrow-down"></i> Télécharger les slides
  </a>
</p>`
```

As cognitive science research reminds us, the perception of charts is a complex cognitive operation that involves grasping quantitative relationships (hierarchy, intensity, etc.) from visual signals.

Innate factors — for example, a better ability to interpret positions rather than colors as quantitative information — and cultural factors — such as familiarity with different forms of data visualization — influence how well a figure is understood.

Stanislas Dehaene's Collège de France lecture on chart perception discusses these issues and traces the history of data visualization.

On this topic, the Insee library organized an exhibition in 2025 showcasing several visualizations that are worth seeing and appear surprisingly modern.

3 Communicating, an opening to app development

The goal of this course is to introduce the main tools and the approach that *data scientists* should adopt when working with various datasets. However, it is becoming increasingly common for *data scientists* to develop and provide interactive applications offering a range of explorations and automated data visualizations. These are more advanced topics than this course covers, but they often serve as an entry point to *data science* for audiences close to *data scientists*, such as *data engineers*, *data analysts*, or statisticians.

We will mention some of the preferred tools for doing this, especially ecosystems related to *web* applications and `Javascript` tools. This need, now fairly standard for *data scientists*, bridges the gap with production deployment, which is the main focus of a third-year ENSAE course designed by Romain Avouac and myself (course website ensae-reproductibilite.github.io/). This current website, for example, is built on this principle using tools that allow `Python` code to be reproducibly executed on standardized servers and then made available through a website.

4 The `Python` ecosystem

Returning to our course, in this section we will present some basic libraries and visualizations in `Python` that provide a good starting point. There are plenty of resources to deepen and advance in the art of visualization, such as this book [3].

4.1 Data visualization packages

The `Python` ecosystem for data visualization is vast and diverse. Entire books could be dedicated to it [4]. `Python` offers numerous libraries to quickly and relatively easily produce data visualizations[1].

The graphical libraries are mainly divided into two families:

- Libraries for **static representations**. These are primarily intended for integration into fixed publications such as PDFs or text documents. We will mainly present `Matplotlib` and `Seaborn`, but there are others emerging, such as `Plotnine`, an adaptation of `ggplot2` to the `Python` ecosystem.
- Libraries for **interactive representations**. These are suited for *web* representations and allow readers to interact with the displayed graphical representation. Libraries offering these features usually rely on `JavaScript`, the web development ecosystem, with an entry point through `Python`. We will primarily discuss `Plotly` and `Folium` in this family, but many other frameworks exist in this field[2].

[1] To be honest, for a long time, `Python` was a bit less enjoyable in this regard compared to `R`, which benefits from the indispensable library `ggplot2`.

Not built on the grammar of graphics, the main graphical library in `Python`, `Matplotlib`, is more cumbersome to use than `ggplot2`.

`seaborn`, which we will present, simplifies graphical representation somewhat, but again, it is difficult to find something more flexible and universal than `ggplot2`.

The library `plotnine` aims to provide a similar implementation to `ggplot` for `Python` users. Its development is worth following.

[2] In this regard, I highly recommend keeping up with data visualization news on the platform `Observable`, which tends to bring together the communities of *dataviz* specialists and data analysts. The library `Plot` could become a new standard in the coming years, a sort of intermediate between `ggplot` and `d3`.

It is entirely possible to create sophisticated visualizations with an end-to-end `Python` workflow since it is a versatile language with a very rich ecosystem. However, `Python` is not a cure-all, and sometimes it can be useful to finalize a perfectly polished product with other languages, such as `JavaScript` for interactive visualizations or `QGIS` for cartographic work. This course will provide the basic tools to quickly and enjoyably produce work, but as the saying goes, the devil is in the details, so one should not insist on using `Python` for every task.

In the realm of visualization, this course takes the approach of exploring a few central libraries through a limited number of examples by replicating charts found on the open data website of the city of Paris. The best training for visualization remains practicing on datasets, so it is recommended to explore the richness of the open data ecosystem to experiment with visualizations.

4.2 Visualization applications

This part of the course focuses on simple synthetic representations. It does not (*yet?*) cover the construction of data visualization applications where a set of graphs update synchronously based on user interactions.

This indeed exceeds the scope of an introductory course, as building these applications requires mastering more complex concepts like the interaction between a *web* page and a server, having some knowledge of `Linux`, etc. The concepts necessary to understand these tools are at the heart of the third-year course "Deploying Data Science Projects" that Romain Avouac and I teach in the third year at ENSAE.

Nevertheless, since data value creation in the form of applications is very common, it is useful, at a minimum, to mention the distinction between static sites and dynamic applications to provide the right approach and point to the appropriate tools. In the world of applications, it is important to distinguish between the *front* (the page visible to the application's users) and the *back office* (the engine that performs actions based on parameters chosen by the user on the page).

There are primarily two paradigms for making these two elements interact. The key difference between these approaches is the servers they rely on. A static site runs on a web server, whereas `Streamlit` relies on a standard *backend* server. The main difference between these two types of servers lies in their function and usage:

- A *web* server is specifically designed to store, process, and deliver web pages (the *front*) to clients. This includes HTML, CSS, JavaScript files, images, etc. Web servers listen for HTTP/HTTPS requests from user browsers and respond by sending the requested data. This doesn't preclude having complex data processing steps or reactivity by embedding `JavaScript` in the application, but `Python` processing steps are done before the application is made available. For `Python` users, there are several static site generators before deployment via hosting on `Github Pages`. The two most common ecosystems are `Quarto Markdown` and `Django`, with the former being simpler to use and maintain than the latter. This site, for example, is built using `Quarto`, which ensures reproducibility of the presented examples and ergonomic, customizable formatting of the results.
- A standard *backend* server is designed to perform operations in response to a *front*, in this case, a *web* page. In the context of an application built with `Python`, this is a server with an appropriate `Python` environment to execute the code required to respond to any action taken by an application user. The code is executed on demand rather than once and for all, as in the previous approach. This paradigm allows for more application complexity but represents an additional challenge during the deployment phase. In the `Python` ecosystem, the two main tools for building such applications are `Streamlit` and `Dash`, with the former being quicker to implement than the latter. More recently, the dominant `R` equivalent ecosystem, `Shiny`, has been adapted for `Python` by `Posit`.

> ℹ Is tkinter still used?

The ecosystems presented above for reactive applications are *web frameworks*. They are distinct from heavier clients like `tkinter`, the historical tool for building graphical user interfaces. Besides the more rudimentary aspect of `tkinter` interfaces compared to those of `Streamlit`, `Dash`, or `Shiny`, there are strong reasons to prefer the latter over `tkinter`.

`Tkinter` is a heavy client, meaning it is tied to an operating system and requires pre-installation of *packages* before the interface can run. While it is certainly possible to make it portable, as discussed in the production course, there are many reasons why this approach may lead to errors or unexpected bugs. *Web frameworks* have the advantage of simplifying this deployment process by separating the *front* (HTML and CSS pages) from the *back* (the `Python` code). They have naturally become more popular, even though many dated online resources still exist for developing applications with `tkinter`.

When it comes to building applications, the first instinct should be: *"Do I need to build a reactive application, or will a static site suffice?"* The latter is much easier to implement and has minimal maintenance overhead, making it a rational choice in many cases. If building a static site becomes complex, for example, due to sophisticated calculations that would be difficult to implement without `JavaScript` skills, you can then consider separating the *front* from the *back* by delegating the calculations to an API, for example, built using `FastAPI`. This can be a practical method to deploy a machine learning model, as will be discussed in the final chapter of the modeling section. If implementing an API seems too complicated or overkill for the task, then you can turn to a reactive application like `Streamlit`.

Again, building an application involves concepts that go beyond an introductory level in `Python`. However, being aware of the right practices can save significant time by avoiding pitfalls due to poor initial choices.

4.3 Summary of this section

Returning to the content of this section after this aside, it is divided into two parts, and each chapter is dual in nature, depending on whether we are focused on static or dynamic representations:

- First, we will discuss standard graphical representations (histograms, bar charts, etc.) to synthesize quantitative information;
  - Static representations will rely on `Pandas`, `Matplotlib`, and `Seaborn`
  - Reactive charts will be built using `Plotly`
- Second, we will present cartographic representations:
  - Static maps created with `Geopandas` or `plotnine`
  - Reactive maps using `Folium` (a `Python` adaptation of the `Leaflet.js` library)

4.4 Useful references

Data visualization is an art that is learned primarily through practice, especially at the beginning. However, it is not always easy to produce readable and ergonomic visualizations, so it is helpful to draw inspiration from examples by specialists (major media outlets offer excellent visualizations).

Here are some useful resources on these topics:

- `Datawrapper` offers an excellent blog on best practices for visualization, particularly with articles by Lisa Charlotte Muth. I especially recommend this article on colors and this one on text;
- The blog of Eric Mauvière;
- *"La Sémiologie graphique de Jacques Bertin a cinquante ans"*;
- The trending visualizations on `Observable`;

- The *New York Times* (masters of *dataviz*) reviews the best visualizations of the year annually, often in the vein of *data scrollytelling*. For example, see the 2022 retrospective.

Some resources on Streamlit or Dash

Besides the 3rd-year course (in French) at ENSAE, Insee's data science lab has built many tutorials to get to grips with the ecosystems of reactive applications in `Python`, which are among the most attractive products of the `Python` ecosystem.

Here, for example, is a very detailed 101 tutorial on `Streamlit` for building a `Yuka`-style application on `openfoodfacts` data. Another step-by-step tutorial built by Insee is devoted to `streamlit` and aims at building an air traffic dashboard.

And a few additional references mentioned in this introduction:

Informations additionnelles

Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
```

```
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

Bibliography

[1] J. Bertin, *Sémiologie graphique*. Paris: Mouton/Gauthier-Villars, 1967.

[2] G. Palsky, “La Sémiologie graphique de Jacques Bertin a cinquante ans,” *Visions carto (en ligne)*, 2017.

[3] C. O. Wilke, *Fundamentals of data visualization: a primer on making informative and compelling figures*. O'Reilly Media, 2019.

[4] K. Dale, *Data Visualization with Python and JavaScript*. " O'Reilly Media, Inc.", 2022.